



Junções e Agregação

Aula 15 · Bloco 4 — SQL Básico

Reunir o que a normalização separou, e resumir o resultado

Nesta aula

1. Por que os dados estão em **duas tabelas**
2. `INNER JOIN`
3. `LEFT JOIN` e o que ele **revela**
4. Contar, somar, medir
5. `GROUP BY`
6. `HAVING`
7. Subconsulta e `VIEW`



A junção é o preço da normalização

E ela é barata.

Você paga **uma linha de SQL**
para reunir o que separou por bons motivos.

O outro caminho seria pagar
com dado contraditório,
todo dia, para sempre.

INNER JOIN

```
SELECT e.id_emprestimo, u.nome, e.data_retirada  
FROM emprestimo e  
JOIN usuario u ON u.matricula = e.matricula  
ORDER BY e.id_emprestimo;
```

Três coisas: o **apelido** (`emprestimo e`), o `ON` que diz **por onde** se ligam, e o prefixo `u.` que tira a ambiguidade.

💡 Se não souber o que escrever no `ON`, volte ao diagrama: a linha entre as tabelas é a resposta.

E não para em duas tabelas

```
SELECT u.nome AS usuario, o.titulo AS obra, x.tombo, e.data_prevista
FROM emprestimo e
JOIN usuario u ON u.matricula = e.matricula
JOIN exemplar x ON x.tombo = e.tombo
JOIN obra o ON o.isbn = x.isbn
WHERE e.data_devolucao IS NULL;
```

Quatro tabelas — porque o empréstimo aponta para o **exemplar**, e é o exemplar que aponta para a obra.

⚠ Esquecer o **ON** produz o produto cartesiano

Cada linha de uma tabela
combinada com **cada** linha da outra.

Com 6 empréstimos e 6 usuários,
36 linhas de lixo.

O sintoma: um resultado grande demais
e visivelmente repetitivo.

LEFT JOIN — o que o INNER JOIN esconde

O acervo tem **cinco** obras. Esta consulta devolve **quatro**:

```
SELECT o.titulo, a.nome AS autor
FROM obra o
JOIN escrita s ON s.isbn = o.isbn
JOIN autor a ON a.id_autor = s.id_autor;
```

"Introdução à Estatística" **sumiu**. Ela existe, tem exemplar na prateleira, e não tem autor cadastrado.

O **INNER JOIN** a descartou em silêncio.

Com **LEFT JOIN**, ela aparece

titulo	autor
Introdução à Estatística	
Redes de Computadores	Andrew Tanenbaum
Redes de Computadores	David Wetherall

E daí sai o uso mais valioso — **encontrar o que não tem par:**

```
LEFT JOIN escrita s ON s.isbn = o.isbn
WHERE s.isbn IS NULL;           -- obras sem nenhum autor
```

Relatório com menos linhas do que você esperava?

Troque o `JOIN` por `LEFT JOIN`
antes de procurar erro
em qualquer outro lugar.

Na maioria das vezes o dado está lá,
e a junção o descartou.

Contar, somar, medir

```
SELECT count(*)           AS total_emprestimos,  
       count(data_devolucao) AS ja_devolvidos  
FROM emprestimo;
```

total_emprestimos	ja_devolvidos
6	2

💡 A **diferença** entre os dois é a resposta de "quantos estão em aberto?" — `count(coluna)` ignora os vazios.

GROUP BY : um resultado por grupo

```
SELECT u.categoria, count(*) AS emprestimos
  FROM emprestimo e
  JOIN usuario u ON u.matricula = e.matricula
 GROUP BY u.categoria;
```

categoria	emprestimos
aluno	5
professor	1

! O erro mais comum de todas as aulas de SQL

```
ERROR: column "u.nome" must appear in the  
GROUP BY clause or be used in an  
aggregate function
```

Se o grupo "aluno" tem cinco empréstimos
de três pessoas, **qual nome** mostrar?

Ou agrupe por ele, ou agregue-o,
ou tire-o da consulta.

HAVING : filtrar grupos, não linhas

```
SELECT o.titulo, count(x.tombo) AS exemplares
FROM obra o
JOIN exemplar x ON x.isbn = o.isbn
GROUP BY o.isbn, o.titulo
HAVING count(x.tombo) > 1;
```

Não dá para escrever `WHERE count(...) > 1` : quando o `WHERE` roda, **ainda não existe contagem nenhuma**.

`FROM` → `WHERE` → `GROUP BY` → `HAVING` → `ORDER BY`

Subconsulta e **VIEW**

```
SELECT u.matricula, u.nome  
FROM usuario u  
WHERE NOT EXISTS (SELECT 1 FROM emprestimo e WHERE e.matricula = u.matricula);  
  
CREATE OR REPLACE VIEW emprestimos_em_aberto AS  
SELECT ... FROM emprestimo e JOIN usuario u ... WHERE e.data_devolucao IS NULL;
```

O atendente consulta a **VIEW** e **nunca precisa saber que são quatro tabelas.**

É a independência lógica da Aula 10, virando comando.

Exercícios da aula

Na pasta `aula-15/` :

1. `ex01.sql` — quatro junções de duas tabelas, com os resultados;
2. `ex02.sql` — duas junções de três ou mais tabelas;
3. `ex03.sql` — cinco resumos com `GROUP BY` , ordenados do maior para o menor;
4. `ex04.sql` — `LEFT JOIN` e `HAVING` , inclusive as obras com **zero** exemplares;
5. **Desafio**  `ex05.sql` — o relatório mensal numa consulta só, e depois como `VIEW` .

Começa aqui o projeto final

Do minimundo ao banco rodando.

Você já tem todas as peças.

Próxima aula: 16 — Revisão e próximos passos